

Raspberry Pi Real-time Secure Server Implementation Guide

A complete, production-ready blueprint for deploying an SSL-secured Node.js real-time WebSocket server infrastructure on a Raspberry Pi targeting **stuffnest.org** and its subdomains.

1. SSL Certificate Automation (Let's Encrypt Certbot)

Execute these commands on your Raspberry Pi to install Certbot and request a wildcard certificate covering both your apex domain and all subdomains using a DNS challenge.

```
# Update system packages
sudo apt update && sudo apt upgrade -y

# Install Certbot
sudo apt install certbot -y

# Request a wildcard certificate via DNS challenge
sudo certbot certonly --manual --preferred-challenges=dns \
-d stuffnest.org \
-d "*.stuffnest.org"\

# Note: Certbot will prompt you to add a TXT record named _acme-challenge.stuffnest.org
# into your DNS management panel before issuing the certificates.
```

2. Server Environment Setup

Initialize a new Node.js project directory and install the necessary dependencies for native secure WebSockets and styling.

```
# Create project architecture
mkdir -p ~/realtime-server && cd ~/realtime-server

# Initialize project environment
npm init -y

# Install production dependencies
npm install ws dotenv

# Create environment configurations and source entrypoints
touch .env server.js index.html
```

3. Environment Variables Configuration (.env)

Define your configuration variables inside the **.env** file. Point the certificate paths directly to your Let's Encrypt live directory.

```
PORT=8443
SSL_PRIVATE_KEY_PATH=/etc/letsencrypt/live/stuffnest.org/privkey.pem
SSL_CERTIFICATE_PATH=/etc/letsencrypt/live/stuffnest.org/fullchain.pem
```

4. Backend Implementation (server.js)

This production script initializes a native Node.js HTTPS server binding your SSL keys, attaches a secure WebSocket (WSS) layer, and sets up event handling.

```
const fs = require('fs');
const https = require('https');
const WebSocket = require('ws');
require('dotenv').config();

// Verify presence of SSL certificates before initialization
try {
  const config = {
    port: process.env.PORT || 8443,
    key: fs.readFileSync(process.env.SSL_PRIVATE_KEY_PATH),
    cert: fs.readFileSync(process.env.SSL_CERTIFICATE_PATH)
  };

  // Create base HTTPS infrastructure layer
  const server = https.createServer({ key: config.key, cert: config.cert }, (req, res) => {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Secure WebSocket Server Infrastructure Running\n');
  });

  // Attach Secure WebSocket (WSS) server layer
  const wss = new WebSocket.Server({ server });

  wss.on('connection', (ws, req) => {
    const clientIp = req.socket.remoteAddress;
    console.log(`[CONNECTED] Client established secure session from internal/external IP: ${clientIp}`);

    // Handle incoming frame payloads
    ws.on('message', (message) => {
      console.log(`[RECEIVED] Raw Frame Payload: ${message}`);

      // Structured response broadcast back to the active socket client
      const responsePayload = JSON.stringify({
        status: "success",
        timestamp: new Date().toISOString(),
        echo: message.toString()
      });
      ws.send(responsePayload);
    });

    ws.on('close', () => {
      console.log(`[DISCONNECTED] Client terminated secure session.`);
    });

    ws.on('error', (error) => {
      console.error(`[SOCKET ERROR] ${error.message}`);
    });
  });

  server.listen(config.port, () => {
    console.log(`[INITIALIZED] Production WSS server operational on port: ${config.port}`);
  });
}
```

```

} catch (error) {
console.error(`[FATAL CRITICAL CRASH] Infrastructure failed to boot: ${error.message}`);
process.exit(1);
}

```

5. Client Testing Code (index.html)

A lightweight client browser setup configured to query the real-time server layer securely across your subdomains via WSS.

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Real-Time Client Test Interface</title>
<style>
body { font-family: -apple-system, sans-serif; margin: 30px; background: #f7fafc; color:
#2d3748; }
#log { background: #1a202c; color: #48bb78; padding: 15px; border-radius: 5px; height:
250px; overflow-y: auto; font-family: monospace; }
</style>
</head>
<body>
<h2>WSS Real-Time Connection Diagnostic Dashboard</h2>
<button onclick="sendMessage()">Dispatch Real-time Event Frame</button>
<h3>Session Event Stream Logging</h3>
<div id="log"></div>

<script>
// Targets a dedicated subdomain mapped directly to your Raspberry Pi infrastructure
const socket = new WebSocket('wss://rt.stuffnest.org:8443');
const logDiv = document.getElementById('log');

socket.onopen = () => log('SUCCESS: Secure TLS session established with Raspberry Pi
server layers.');
```

```

socket.onclose = () => log('NOTICE: TLS connection closed safely.');
```

```

socket.onerror = (err) => log('ERROR: Connection rejected or handshake failure.');
```

```

socket.onmessage = (event) => log('RECEIVED: ' + event.data);

function sendMessage() {
const payload = "Hello from stuffnest client endpoint!";
socket.send(payload);
log('SENT: ' + payload);
}

function log(msg) {
logDiv.innerHTML += ` [&${null}#43;] ${msg}<br/>`;
logDiv.scrollTop = logDiv.scrollHeight;
}
</script>
</body>
</html>

```

6. Process Management Configuration (systemd)

Ensure the process auto-starts on system boots and handles background execution logging continuously.

```
# Create your background process monitor configuration file
sudo nano /etc/systemd/system/realtime-server.service

# Paste the configuration architecture below inside:
[Unit]
Description=Raspberry Pi Real-time Secure Server Engine
After=network.target

[Service]
Type=simple
User=root
WorkingDirectory=/root/realtime-server
ExecStart=/usr/bin/node server.js
Restart=on-failure

[Install]
WantedBy=multi-user.target

# Reload and boot up configurations system wide
sudo systemctl daemon-reload
sudo systemctl enable realtime-server.service
sudo systemctl start realtime-server.service
```